

legged_mppi: Whole-Body Locomotion MPPI Mathematical Formulation and Implementation Analysis

Notes based on analysis of the `legged_mppi` codebase

July 20, 2026

Contents

1	Scope and Executive Summary	3
2	Robot Model, State, and Input	3
2.1	Degrees of Freedom and Joint Order	3
2.2	Observation State	4
2.3	Control Input and Low-Level Actuation	4
3	Prediction Dynamics and Contact	4
3.1	Discrete MuJoCo Dynamics	4
3.2	Parallel Rollout Execution	5
3.3	Contact Treatment	5
4	Finite-Horizon Stochastic Optimization Problem	5
5	Reference Generation and Gait Scheduling	6
5.1	Waypoint-Level Body Reference	6
5.2	Periodic Joint-Space Gait	6
5.3	Complete State Reference	7
6	Sampling and MPPI-Style Update	7
6.1	Warm Start	7
6.2	Cubic Gaussian Sampling	7
6.3	Cost Normalization and Exponential Weights	8
7	Cost Function	8
7.1	Quaternion Error	8
7.2	Position and State Tracking	8
7.3	Virtual PD-Effort Cost	9
7.4	Stage and Rollout Cost	9
8	Waypoint and Task Progression	9
9	Complete Receding-Horizon Algorithm	10
10	Default Configuration Summary	10
11	Comparison with OCS2 Legged MPC	11
12	Implementation Caveats and Limitations	11

13 Recommended Validation	12
14 Pipeline Summary	12

1 Scope and Executive Summary

This document analyzes the locomotion controller implemented by `legged_mppi`. The presentation follows the problem-oriented style of the accompanying OCS2 legged-MPC note, but the mathematical description here is derived from the actual Python, YAML, gait, and MuJoCo files in this repository. The primary implementation sources are:

- `control/controllers/base_controller.py`,
- `control/controllers/mppi_locomotion.py`,
- `control/gait_scheduler/scheduler.py`,
- `utils/tasks.py` and the `mppi_gait_config_*.yaml` files,
- the Go1 MJCF files under `models/go1`, and
- `interface/simulator.py` and `scripts/run_mppi_locomotion.py`.

The main architectural choices are:

1. The controller uses the complete 37-dimensional floating-base Go1 state rather than a reduced centroidal state.
2. The optimized input is a sequence of 12 desired joint positions, not joint torques, joint velocities, or contact forces.
3. Nonlinear rigid-body dynamics and contact are evaluated by parallel MuJoCo rollouts. No analytical dynamics derivatives are required.
4. A periodic, precomputed 24-row gait file supplies desired joint positions and velocities over the prediction horizon.
5. Each rollout is scored by body-state tracking, gait tracking, and a virtual PD-effort penalty.
6. The implemented update is an MPPI-style exponentially weighted mean of sampled absolute action sequences. It is not the canonical path-integral noise-increment update.

Contact is therefore *implicit in the rollout model*. The gait file is a kinematic motion prior; it does not impose a hard stance/swing contact-mode schedule. Feasibility is produced by action clipping, MuJoCo joint and force limits, and the simulated contact dynamics rather than explicit OCP constraints.

2 Robot Model, State, and Input

2.1 Degrees of Freedom and Joint Order

The locomotion model is a floating-base Unitree Go1 with twelve actuated joints. Its MuJoCo dimensions are

$$n_q = 19, \quad n_v = 18, \quad n_u = 12. \quad (1)$$

The actuator order used consistently by the controller is

$$\mathbf{q}_j = [q_{FR,h}, q_{FR,t}, q_{FR,c}, q_{FL,h}, q_{FL,t}, q_{FL,c}, \\ q_{RR,h}, q_{RR,t}, q_{RR,c}, q_{RL,h}, q_{RL,t}, q_{RL,c}]^T, \quad (2)$$

where h, t, and c denote hip abduction, thigh, and calf joints, respectively.

2.2 Observation State

The controller observation concatenates MuJoCo `qpos` and `qvel`:

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_b \\ \mathbf{q}_b \\ \mathbf{q}_j \\ \mathbf{v}_b \\ \boldsymbol{\omega}_b \\ \dot{\mathbf{q}}_j \end{bmatrix} \in \mathbb{R}^{37}. \quad (3)$$

Here $\mathbf{p}_b \in \mathbb{R}^3$ is the floating-base position in the world frame, $\mathbf{q}_b = [q_w, q_x, q_y, q_z]^\top \in \mathbb{S}^3$ is its quaternion, $\mathbf{q}_j \in \mathbb{R}^{12}$ is the joint position, $\mathbf{v}_b$ and $\boldsymbol{\omega}_b$ are base linear and angular velocities, and $\dot{\mathbf{q}}_j \in \mathbb{R}^{12}$ contains joint velocities.

Python slice	Dimension	Quantity
0:3	3	base position $\mathbf{p}_b$
3:7	4	base quaternion $\mathbf{q}_b$
7:19	12	joint position $\mathbf{q}_j$
19:22	3	base linear velocity $\mathbf{v}_b$
22:25	3	base angular velocity $\boldsymbol{\omega}_b$
25:37	12	joint velocity $\dot{\mathbf{q}}_j$

MuJoCo’s FULLPHYSICS rollout state additionally stores time:

$$\mathbf{x}^{\text{full}} = [t, \mathbf{q}_{\text{pos}}^\top, \mathbf{q}_{\text{vel}}^\top]^\top \in \mathbb{R}^{38}. \quad (4)$$

The implementation removes the first element before evaluating cost.

2.3 Control Input and Low-Level Actuation

The optimized input is a desired joint-position vector:

$$\mathbf{u} = \mathbf{q}_{j,\text{des}} \in \mathbb{R}^{12}, \quad \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}. \quad (5)$$

The limits are repeated once per leg:

$$\mathbf{u}_{\min}^{\text{leg}} = [-0.863, -0.686, -2.818]^\top, \quad \mathbf{u}_{\max}^{\text{leg}} = [0.863, 4.501, -0.888]^\top. \quad (6)$$

In the default rollout MJCF, an affine position actuator approximately applies

$$\tau_i = \text{operatornamesat}_{[\tau_i^-, \tau_i^+]} (55(u_i - q_{j,i}) - 3\dot{q}_{j,i}). \quad (7)$$

The nominal force range is $[-23.7, 23.7]$ N m, enlarged to $[-35.55, 35.55]$ N m for the calf. The hardware interface sends the same desired positions in Unitree position-control mode with $K_p = 55$, $K_d = 3$, zero desired velocity, and zero feedforward torque.

3 Prediction Dynamics and Contact

3.1 Discrete MuJoCo Dynamics

For analysis, one rollout step can be written as

$$\mathbf{x}_{k+1} = F_{\text{MJ}}(\mathbf{x}_k, \mathbf{u}_k; \mathcal{M}, \Delta t, \boldsymbol{\eta}), \quad (8)$$

where $\mathcal{M}$ is the MJCF robot/environment model and $\boldsymbol{\eta}$ collects numerical and contact parameters. The default values include

$$\Delta t = 0.01 \text{ s}, \quad \text{o_solref} = [0.02, 1.0], \quad (9)$$

with pyramidal friction cones in the prediction model.

Equation (8) includes floating-base multibody dynamics, actuator saturation, joint limits, gravity, collision detection, normal contact, and friction. Unlike the SRBD model in the OCS2 note, the implementation does not expose contact force as an optimization variable and does not explicitly write a centroidal momentum equation.

For each controller update, all N candidates begin from the repeated current observation $\hat{\mathbf{x}}_t$ and are integrated for H steps. The output tensor has shape

$$N \times H \times 38, \quad (10)$$

and is evaluated as $N \times H \times 37$ after removing time.

3.2 Parallel Rollout Execution

Candidates are divided into chunks and submitted to a thread pool. Every worker owns a thread-local `MjData`, while all workers share the immutable `MjModel`. The MuJoCo rollout API performs the H -step integrations. For the default configuration this means

$$NH = 30 \times 40 = 1200 \quad (11)$$

contact-dynamics steps per update, distributed over five workers.

3.3 Contact Treatment

The controller does not optimize a binary contact mode σ_k , and the gait file does not activate mode-dependent constraints. Instead,

$$\text{desired joint gait} \longrightarrow \text{foot motion} \longrightarrow \text{MuJoCo collision/contact response}. \quad (12)$$

Thus, stance and swing emerge from tracking a periodic joint-space reference inside the simulated contact dynamics. Contact timing can deviate from the nominal gait when sampled actions, terrain, or tracking errors change the motion.

The prediction and closed-loop simulator normally load separate MJCF files. For the default walking task these are `go1_scene_mppi_pyr.xml` for rollouts and `go1_scene_mppi.xml` for execution. They share the same Go1 structure and actuator convention, but the prediction variant explicitly selects pyramidal contact. Any further mismatch in geometry, contacts, or parameters acts as model error.

4 Finite-Horizon Stochastic Optimization Problem

At update time t , let

$$\mathbf{U} = [\mathbf{u}_0, \dots, \mathbf{u}_{H-1}] \in \mathbb{R}^{H \times 12} \quad (13)$$

be the warm-start action sequence. The controller samples N bounded candidate sequences $\mathbf{U}^{(n)}$ and approximately solves

$$\begin{aligned} \min_{\mathbf{U}} \quad & S(\mathbf{U}; \hat{\mathbf{x}}_t, \mathbf{X}_t^{\text{ref}}) = \sum_{k=0}^{H-1} \ell(\mathbf{x}_{k+1}, \mathbf{u}_k, \mathbf{x}_k^{\text{ref}}), \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = F_{\text{MJ}}(\mathbf{x}_k, \mathbf{u}_k), \\ & \mathbf{x}_0 = \hat{\mathbf{x}}_t, \\ & \mathbf{u}_{\min} \leq \mathbf{u}_k \leq \mathbf{u}_{\max}. \end{aligned} \quad (14)$$

There is no separately weighted terminal cost in the locomotion implementation. The rollout sum includes all returned states with the same stage-cost expression.

This is a sample-based numerical optimization, not a direct transcription of the dynamics. States are never independent decision variables, and dynamics or cost Jacobians are not computed.

5 Reference Generation and Gait Scheduling

5.1 Waypoint-Level Body Reference

For waypoint g , the task dictionary defines a desired position, nominal orientation, commanded planar velocity, distance threshold, gait name, and waiting time. The controller stores

$$\mathbf{r}_b^{\text{ref}} = \begin{bmatrix} \mathbf{p}_g \\ \mathbf{q}_g \\ v_{x,g}^{\text{cmd}} \\ v_{y,g}^{\text{cmd}} \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \in \mathbb{R}^{13}. \quad (15)$$

The final four zeros correspond to desired vertical linear velocity and three angular-velocity components.

When the position error exceeds 0.1 m and the waypoint timer is not in its waiting state, $\mathbf{q}_g$ is recomputed to point toward the goal. If $\mathbf{d} = \mathbf{p}_g - \mathbf{p}_b$, then

$$\psi_g = \text{atan2}(d_y, d_x), \quad (16)$$

$$\theta_g = -\text{atan2}\left(d_z, \sqrt{d_x^2 + d_y^2}\right), \quad (17)$$

$$\mathbf{q}_g = \mathbf{q}_z(\psi_g) \otimes \mathbf{q}_y(\theta_g). \quad (18)$$

Otherwise, the desired orientation is the identity quaternion.

5.2 Periodic Joint-Space Gait

Each gait is loaded from a tab-separated matrix

$$\mathbf{G} \in \mathbb{R}^{24 \times L}. \quad (19)$$

The first twelve rows are joint-position references and the last twelve rows are joint-velocity references:

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_q \\ \mathbf{G}_{\dot{q}} \end{bmatrix}. \quad (20)$$

The `in_place` and `walk_fast` files have $L = 100$ columns, whereas the two medium-speed files used by `walk` and `trot` have $L = 74$. At controller phase s , the reference horizon is selected cyclically:

$$\mathbf{G}_{s,H} = [\mathbf{G}_{:,s}, \mathbf{G}_{:,s+1}, \dots, \mathbf{G}_{:,s+H-1}], \quad (21)$$

where column indices wrap modulo L . After every MPPI update, the index array is rolled by one column.

The active scheduler is selected from

$$\{\text{in_place}, \text{walk}, \text{walk_fast}, \text{trot}\} \quad (22)$$

when the waypoint changes. Each scheduler is a persistent object, so reselecting a gait resumes its previously stored phase rather than automatically resetting phase zero.

5.3 Complete State Reference

For rollout step k , the reference used by the cost is

$$\mathbf{x}_k^{\text{ref}} = \begin{bmatrix} \mathbf{p}_g \\ \mathbf{q}_g \\ \mathbf{G}_{q,s+k} \\ v_{x,g}^{\text{cmd}} \\ v_{y,g}^{\text{cmd}} \\ 0 \\ \mathbf{0}_3 \\ \mathbf{G}_{\dot{q},s+k} \end{bmatrix} \in \mathbb{R}^{37}. \quad (23)$$

Before cost evaluation, the predicted world-frame base linear velocity is rotated into the predicted body frame:

$${}^B \mathbf{v}_{b,k} = R(\mathbf{q}_{b,k})^\top {}^W \mathbf{v}_{b,k}. \quad (24)$$

Accordingly, the commanded x and y velocities in Eq. (23) are interpreted in the body frame. Base angular velocity is not transformed by this cost-side operation.

6 Sampling and MPPI-Style Update

6.1 Warm Start

At initialization the entire horizon is filled with

$$\mathbf{u}_{\text{init}} = \mathbf{1}_2 \otimes [-0.3, 1.34, -2.83, 0.3, 1.34, -2.83]^\top, \quad (25)$$

which produces the twelve entries for the four legs. After an update, the new plan is shifted:

$$\mathbf{u}_{k,t+1}^{\text{nom}} = \begin{cases} \mathbf{u}_{k+1,t}^+, & k = 0, \dots, H-2, \\ \mathbf{u}_{H-1,t}^+, & k = H-1. \end{cases} \quad (26)$$

The last optimized action is therefore held at the tail.

6.2 Cubic Gaussian Sampling

The default sampler uses $K = 4$ knot indices. For $H = 40$, rounded linearly spaced indices give

$$\mathcal{K} = \{0, 13, 26, 39\}. \quad (27)$$

For sample n , independent Gaussian knot noise is generated as

$$\boldsymbol{\epsilon}_{n,j} \sim \mathcal{N}(\mathbf{0}, \text{diag}(\boldsymbol{\sigma}^2)), \quad j \in \mathcal{K}. \quad (28)$$

The default walking standard deviation repeats

$$\boldsymbol{\sigma}^{\text{leg}} = [0.06, 0.1, 0.1]^\top \quad (29)$$

for all four legs. During a **trot** task phase, the controller changes this to $[0.06, 0.2, 0.2]^\top$ per leg. A cubic spline interpolates the noisy knot actions across the full horizon:

$$\tilde{\mathbf{u}}_k^{(n)} = \text{CubicSpline}(\{\mathbf{u}_j^{\text{nom}} + \boldsymbol{\epsilon}_{n,j}\}_{j \in \mathcal{K}})(k). \quad (30)$$

The actual candidate is clipped componentwise:

$$\mathbf{u}_k^{(n)} = \text{clip}(\tilde{\mathbf{u}}_k^{(n)}, \mathbf{u}_{\min}, \mathbf{u}_{\max}). \quad (31)$$

The base class also supports independent normal noise at every time step, but all inspected locomotion configurations select cubic sampling.

6.3 Cost Normalization and Exponential Weights

Let S_n be the total rollout cost. The implementation calculates

$$S_{\min} = \min_n S_n, \quad S_{\max} = \max_n S_n, \quad (32)$$

then assigns the unnormalized weight

$$\tilde{w}_n = \exp \left[-\frac{1}{\lambda} \frac{S_n - S_{\min}}{S_{\max} - S_{\min}} \right]. \quad (33)$$

The new plan is the weighted average of the *absolute candidates*:

$$\mathbf{u}_k^+ = \text{clip} \left(\frac{\sum_{n=1}^N \tilde{w}_n \mathbf{u}_k^{(n)}}{\sum_{n=1}^N \tilde{w}_n + 10^{-10}}, \mathbf{u}_{\min}, \mathbf{u}_{\max} \right). \quad (34)$$

The default temperature is $\lambda = 0.1$ in simulation and 0.15 in the walking hardware configuration.

Equation (34) is important for interpreting the implementation. Canonical MPPI is commonly expressed as a weighted perturbation update $\mathbf{U}^+ = \mathbf{U} + \sum_n w_n \mathbf{E}^{(n)}$, together with a cost/noise correction derived from stochastic optimal control. Here the implementation instead uses a min-max normalized ranking and directly averages bounded sampled actions. It is best described as an MPPI-style sampling controller.

7 Cost Function

7.1 Quaternion Error

For predicted and reference quaternions, the scalar sign-invariant error is

$$d_q(\mathbf{q}, \mathbf{q}^{\text{ref}}) = 1 - \left| \mathbf{q}^\top \mathbf{q}^{\text{ref}} \right|. \quad (35)$$

The implementation writes this same scalar into all four quaternion-error slots. Hence the effective orientation cost is

$$\left(\sum_{j=3}^6 Q_{jj} \right) d_q^2. \quad (36)$$

In the default walking configuration, only $Q_{33} = 400000$ is nonzero in that block, so the orientation contribution is $400000 d_q^2$.

7.2 Position and State Tracking

The original base-position error is removed from the quadratic state error. Position is instead penalized with a weighted L_1 term:

$$\ell_p(\mathbf{x}) = Q_{00}|p_x - p_x^{\text{ref}}| + Q_{11}|p_y - p_y^{\text{ref}}| + Q_{22}|p_z - p_z^{\text{ref}}|. \quad (37)$$

Let $\mathbf{e}_x$ equal $\mathbf{x} - \mathbf{x}^{\text{ref}}$, with entries 0:3 set to zero and entries 3:7 replaced by the scalar error in Eq. (35). The remaining state cost is

$$\ell_x = \mathbf{e}_x^\top \mathbf{Q} \mathbf{e}_x + \ell_p. \quad (38)$$

Thus ℓ_x includes orientation, gait joint-position tracking, body velocity tracking, angular-velocity regulation, and gait joint-velocity tracking.

For the default walking task, the notable diagonal weights are

$$\begin{aligned}
\mathbf{Q}_p &= \text{diag}(300, 300, 500), \\
Q_{q, \text{ori}} &= 400000, \\
\mathbf{Q}_{q_j} &= 8000 \mathbf{I}_{12}, \\
\mathbf{Q}_{v_b} &= \text{diag}(1, 10^{-4}, 10^{-4}), \\
\mathbf{Q}_{\omega_b} &= 10^{-4} \mathbf{I}_3, \\
\mathbf{Q}_{\dot{q}_j} &= 0.1 \mathbf{I}_{12}.
\end{aligned} \tag{39}$$

These values show that the gait posture and body orientation dominate the quadratic reference tracking, while forward-speed error receives a much smaller weight.

7.3 Virtual PD-Effort Cost

The cost does not penalize desired position $\mathbf{u}$ directly. It constructs a virtual PD effort

$$\mathbf{e}_u = 50(\mathbf{u} - \mathbf{q}_j) - 3\dot{\mathbf{q}}_j \tag{40}$$

and applies

$$\ell_u = \mathbf{e}_u^\top \mathbf{R} \mathbf{e}_u, \quad \mathbf{R} = 10^{-3} \mathbf{I}_{12} \tag{41}$$

in the default walking configuration. This is a proxy for actuator effort. It differs slightly from the rollout actuator in Eq. (7), which uses proportional gain 55 and force saturation.

7.4 Stage and Rollout Cost

The stage cost implemented by `quadruped_cost_np` is

$$\boxed{\ell(\mathbf{x}, \mathbf{u}, \mathbf{x}^{\text{ref}}) = \mathbf{e}_x^\top \mathbf{Q} \mathbf{e}_x + \mathbf{e}_u^\top \mathbf{R} \mathbf{e}_u + \ell_p.} \tag{42}$$

The rollout cost is the un-discounted sum

$$S_n = \sum_{k=0}^{H-1} \ell(\mathbf{x}_{k+1}^{(n)}, \mathbf{u}_k^{(n)}, \mathbf{x}_k^{\text{ref}}). \tag{43}$$

No explicit slip, contact-force, friction-cone, foot-clearance, self-collision, control-rate, or terminal term appears in this locomotion cost. Their effects are represented only indirectly through dynamics and state/gait tracking.

8 Waypoint and Task Progression

Tasks such as `walk_straight`, `walk_octagon`, `stairs`, and `big_box` define ordered waypoint sequences. The active waypoint is considered geometrically reached when

$$\|\mathbf{p}_g - \mathbf{p}_b\| < r_g, \tag{44}$$

where the inspected tasks normally use $r_g = 0.2$ m. The simulator and hardware loop call `next_goal()` only while this condition holds.

The timer is measured in controller calls rather than seconds. At the nominal 100 Hz rate, 100 timer increments correspond to approximately one second. When the dwell count completes, `next_goal()` updates the goal position, planar velocity command, and active gait. At the last goal it sets the task-success flag.

The timer does not enforce continuous residence inside the goal region: if the robot leaves the threshold, accumulated elapsed counts are retained until the next successful invocation. This detail matters for interpreting `waiting_times` as accumulated in-threshold calls rather than a strict continuous dwell interval.

9 Complete Receding-Horizon Algorithm

Algorithm 1 legged_mppi locomotion update

Require: Current state $\hat{\mathbf{x}}_t$, previous plan $\mathbf{U}_t$, active gait and waypoint

- 1: Compute the goal direction and update the desired body quaternion.
 - 2: Read the next H cyclic gait columns to form $\mathbf{X}_t^{\text{ref}}$.
 - 3: Add Gaussian noise at four knots, interpolate cubic splines, and clip N candidate action sequences.
 - 4: Repeat $\hat{\mathbf{x}}_t$ as the initial state of all candidates.
 - 5: Roll out all N sequences for H MuJoCo steps in parallel.
 - 6: Rotate predicted base linear velocities into their local body frames.
 - 7: Evaluate S_n from position, state, gait, and virtual-effort costs.
 - 8: Min-max normalize S_n and compute exponential weights.
 - 9: Compute $\mathbf{U}_t^+$ as the weighted mean of absolute candidates.
 - 10: Shift $\mathbf{U}_t^+$ by one step and hold its tail for warm start.
 - 11: Advance the gait phase by one column.
 - 12: **return** First desired joint-position command $\mathbf{u}_{0,t}^+$
-

In simulation the first action is applied for one 10 ms MuJoCo step before the process repeats. In hardware, the first action is published as twelve Unitree motor position commands at a nominal 100 Hz ROS loop rate.

10 Default Configuration Summary

Parameter	Default walking value
Robot	Unitree Go1, floating base
State / input dimension	37 / 12
Input interpretation	desired joint position
Time step / horizon	0.01 s / 40 steps (0.4 s)
Samples / workers	30 / 5
Temperature	0.1 simulation; 0.15 walking hardware
Sampler	cubic spline, four noisy knots
Random seed	42
Nominal gait lengths	74 or 100 samples
Contact parameters	<code>o_solref</code> = [0.02, 1.0]
Action limits	Go1 joint-position limits
Initial plan	fixed crouched joint posture

The YAML files for stairs, large-box traversal, and hardware reuse the same basic solver dimensions while changing cost weights, temperature, noise, or the environment model. Task dictionaries additionally select waypoint, velocity, gait, and model sequences.

11 Comparison with OCS2 Legged MPC

Aspect	OCS2 legged MPC	<code>legged_mppi</code> locomotion
Dynamics	Centroidal/SRBD or full centroidal	Full MuJoCo multibody rollout
State	Centroidal momentum, base pose, joints	MuJoCo floating-base position and velocity state
Optimized input	Contact forces and joint velocities	Desired joint positions
Contact timing	Prescribed mode schedule	Joint-space gait prior plus simulated collision
Contact force	Explicit decision variable	Internal MuJoCo result
Constraints	Mode-dependent equalities, inequalities, barriers	Action clipping, model limits, contact dynamics
Solver	SQP, DDP, or IPM	Sampling and exponential weighted averaging
Derivatives	Analytical/automatic derivatives	Derivative-free rollout scoring
Low-level output	Whole-body controller maps MPC output to torque	Position command sent to simulated or hardware PD servo

The two controllers therefore place complexity in different layers. OCS2 uses an explicit reduced-order OCP and constraint set, followed by a whole-body controller. `legged_mppi` places the full robot, actuator, terrain, and contact response inside the rollout and searches directly in joint-command space.

12 Implementation Caveats and Limitations

1. **The update differs from canonical MPPI.** Cost is normalized by the current batch range, and absolute candidate actions are averaged. The temperature therefore controls selection over relative batch cost rather than cost in its original physical units.
2. **Equal rollout costs are numerically unsafe.** Equation (33) divides by $S_{\max} - S_{\min}$ without an epsilon. If every candidate has the same cost, the weights can become non-finite even though the final weight sum contains 10^{-10} .
3. **There is no explicit nominal candidate.** Every candidate receives random knot noise. Consequently, the current warm-start plan is not guaranteed to be evaluated unchanged.
4. **Gait timing is not a contact constraint.** The desired stance/swing behavior can be lost when the joint-space reference is not tracked. Conversely, very large joint weights can make the controller behave mainly as a gait playback system.
5. **The cost omits several locomotion safety quantities.** There is no direct penalty or constraint for foot slip, ground-reaction force, friction margin, support polygon, body collision, or joint torque.
6. **Virtual and applied effort differ.** The cost uses $K_p = 50$ and no saturation, while the default MuJoCo actuator uses $K_p = 55$ and force limits. Hardware dynamics, latency, and saturation add further mismatch.
7. **Prediction and execution models can differ.** Separate MJCF paths are loaded for the controller and simulator. Contact geometry and solver settings should be audited together for each task.

8. **Velocity-frame consistency requires validation.** The rollout cost explicitly converts world linear velocity into a body frame. The hardware callback also transforms odometry velocity, but its use of the forward rotation should be checked against the frame convention of each odometry source.
9. **Real-time performance is not guaranteed by the nominal rate.** A 100 Hz loop provides only 10 ms per update. The wall-clock time for 1200 contact steps, allocation, interpolation, and cost calculation must be measured on the target CPU.

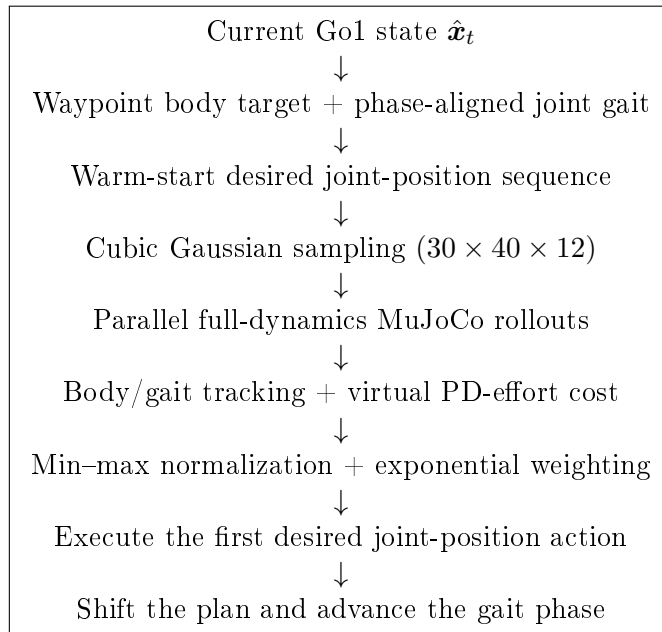
13 Recommended Validation

The following measurements connect the formulation to actual closed-loop behavior:

1. mean, 95th, and 99th percentile MPPI update time and 10 ms deadline miss rate,
2. distribution of $S_{\max} - S_{\min}$, effective sample size, and the fraction of non-finite weights,
3. base position, height, quaternion error, and body-frame velocity error,
4. joint/gait tracking error, actuator saturation rate, and measured effort,
5. foot contact sequence, ground-reaction force, slip velocity, and body collision events,
6. one-step prediction error between the rollout model and simulator or hardware observation, and
7. waypoint residence time, transition count, and task-completion rate.

Useful ablations include normal versus cubic noise, the current absolute-action average versus a perturbation update, inclusion of a noise-free candidate, removal of min-max normalization, varying the gait-state weights, and using identical versus deliberately mismatched prediction/execution contact models.

14 Pipeline Summary



The defining design choice is to optimize desired joint-position trajectories through full MuJoCo contact dynamics. A periodic gait supplies a strong motion prior, while sampling searches

locally for whole-body actions that reduce pose, velocity, gait, and virtual-effort cost in receding-horizon feedback.